

Aliah University

End Semester Examination (Autumn Semester) 2025

2nd Year 3rd Semester 2 Year MCA Programme

Paper Name: Compiler Design

Paper Code: MCAPGPC19

Full Marks: 80

Time: 3 hrs

Group A

(Answer any 10 questions)

10×2=20

1. What are the main differences between compiler and interpreter?
2. What is left recursion? Give one method to eliminate left recursion?
3. What is ambiguity in a grammar? Give an example of an ambiguous construct?
4. What is left factoring? Why is it used in predictive parsing?
5. Compute FIRST(X) for the grammar:
 $X \rightarrow YZ$
 $Y \rightarrow y | \epsilon$
 $Z \rightarrow z$
6. Eliminate left recursion from the grammar: $A \rightarrow Aa | Abc | dAe | f | g$
7. Write any two differences between LL(1) and LR(1) parsers.
8. Generate three-address code (TAC) for: $x = (p-q)+r*s$
9. Give one example each of a syntactic error and a semantic error.
10. Construct a simple NFA for recognizing identifiers that start with a letter followed by digits.
11. Eliminate non-determinism for the grammar: $S \rightarrow bSSaaS | bSsaSb | bSb | a$
12. What is the role of symbol table in a compiler?

PDF Prepared by
Md Mim Akhtar
+91 80018 23387

Group B

(Answer any 6 questions)

6×5=30

1. For the grammar below, compute FIRST and FOLLOW sets for all non-terminals:
 $S \rightarrow AB$
 $A \rightarrow aA | \epsilon$
 $B \rightarrow bB | c$
Also comment on whether the grammar can be parsed using LL(1). [2+2+1]
2. Eliminate left recursion and then perform left factoring on the grammar:
 $S \rightarrow Aa | Ab | c$
 $A \rightarrow Sd | Se | f$ [3+2]
3. Construct the predictive parsing table for the following grammar and state whether it is LL(1) or not:
 $S \rightarrow aA | b$
 $A \rightarrow cS | \epsilon$ [4+1]
4. For the following grammar, construct the syntax tree and the leftmost derivation for the string: $(id + num) * id + num * (id + id)$
Grammar:
 $E \rightarrow E + T | T$
 $T \rightarrow T * F | F$
 $F \rightarrow (E) | id | num$ [3+2]
5. Discuss with an example, different types of intermediate code forms:
i) Postfix notation ii) syntax trees iii) three-address code (TAC) [1+2+2]

L.A
S.A
S.A
C.O.
T.C.G

6. Construct the LR(0) items and canonical collection for the grammar:
 $S \rightarrow A, A \rightarrow aA \mid b$
 Then build the LR(0) automaton structure. [3+2]
7. Generate the three-address code (TAC), quadruples, and triples for the expression:
 $X = (a + b) * (c - d) / e$ [2+2+1]
8. Define activation tree and activation record. For the following program, draw the activation tree when main() is executed: [2+2+1]
- ```
main() {
 A();
 B();
}
A() {
 C();
}
B() {
 C();
 D();
}
```

PDF Prepared by  
**Md Mim Akhtar**  
**+91 80018 23387**

### Group C

(Answer any 3 questions)

3×10=30

- Depict the phases of a compiler with a neat block diagram.  
 Using the input statement,  $A[i + 1] = B[i] + 10$   
 Illustrate how the expression is processed by each phase of the compiler. [4+6]
- Using the McNaughton–Yamada–Thompson construction, build an NFA for the regular expression,  $r = (a|b)(a|b)^*a$   
 Convert this NFA into an equivalent DFA using subset construction algorithm. [5+5]
- What is lexical analysis? Describe tokens, lexemes, and patterns with suitable examples.
  - Why is the lexical analyzer kept separate from the syntax analyzer? [(2+3)+5]
- Given the grammar:  
 $S \rightarrow L = R \mid R$   
 $L \rightarrow * R \mid id$   
 $R \rightarrow L$ 
  - Construct the CLR(1) item sets for this grammar.
  - Merge CLR(1) states to form the LALR(1) item sets. [7+3]
- For the grammar  
 $S \rightarrow AB$   
 $A \rightarrow aA \mid a$   
 $B \rightarrow CB \mid C$   
 $C \rightarrow d \mid \epsilon$ 
  - Compute the FIRST and FOLLOW sets for all non-terminals S, A, B, and C.
  - Using these sets, construct the complete LL(1) parsing table for the grammar and verify whether it is LL(1). [5+5]